



Título del documento a varias líneas

Adrián Durán Durán



Índice

1. Parte I:	3
1.1. Actv 1	3
1.1.1. Definiciones	3
1.1.2. Fases de un programa	4
1.1.3 Ejemplos de conceptos	6
1.2 Actv 2.	8
1.2.1 Los lenguajes de Bajo Nivel hablan directamente el idioma del procesador o están a un solo paso de él.	8
1.2.2. Lenguajes de Nivel Medio	9
1.2.3. Lenguajes de Nivel Alto	9
2. Parte II:	10
2.1. Actv.	10
2.1.1. Fragmento 1	10
2.1.2. Fragmento 2	11
2. 2.1.3. Fragmento 3	11
3. 2.1.4. Fragmento 4	11
2.2. Actv 2.	12
2.2.1 Comparación:	12
WEBGRAFÍA:	13
Pictografía:	13

Nota: Este pdf se ha realizado sin poder acceder todavía al aula virtual, dicho de otra forma: no creo que este bien explicado sinceramente



1. Parte I:

1.1. Actv 1

1.1.1. Definiciones

Describe en tus propias palabras qué es el código fuente, código objeto y código ejecutable.

Código Fuente: Es un archivo de texto que contiene el código escrito en un lenguaje de programación legible por humanos. Este código está escrito en un lenguaje de programación específico **C, C++, Java, Python, JavaScript**, entre otros.

Características:

1. Lo entiende el programador.
2. No se puede ejecutar (son solo palabras).
3. Es un formato de texto (ej: `print("Hola")`).

Código objeto: es una versión intermedia del código fuente que ha sido compilada a un formato binario, pero aún no es ejecutable directamente. Es el resultado de la compilación del código fuente antes de convertirse en código ejecutable.

Características:

1. Nadie lo entiende (ni el programador, ni la máquina).
2. No se puede ejecutar, está incompleto.
3. Es un formato binario (ceros y unos sueltos).



Código ejecutable: es un archivo binario que puede ser ejecutado directamente por el sistema operativo. Este es el resultado final de la compilación (y en algunos casos, el enlace) del código fuente.

Características:

1. Lo entiende el procesador del pc.
2. Se puede ejecutar, funciona por sí solo.
3. Es un formato de archivo final (ej. `.exe` o aplicación instalada).

1.1.2. Fases de un programa

Explica las fases por las que pasa un programa desde que es escrito en un lenguaje de programación hasta que es ejecutado en el procesador.

1. Código Fuente

Escribes las instrucciones del programa en un **lenguaje de programación** (como C, C++ o Rust) usando un editor de texto. Este archivo de texto contiene palabras legibles para los humanos.

2. Preprocesamiento

El **preprocesador** lee el código fuente antes de traducirlo. Busca directivas especiales (como `#include` en C) para incluir bibliotecas o expandir macros. Produce un archivo limpio listo para el compilador.



Sinónimo de preprocesamiento: Limpieza y Calidad de Datos

Depuración de datos: Proceso de detectar y corregir o eliminar registros corruptos o inexactos.

Limpieza de datos (o *Data Cleaning*): Eliminación de ruido, valores nulos o duplicados.

Saneamiento de datos: Acción de limpiar la información para dejarla estructurada y segura.

3. Compilación

El **compilador** traduce el código preprocesado a **lenguaje ensamblador** (código de bajo nivel específico para la arquitectura). Revisa que la gramática y la lógica del código estén correctas según las reglas del lenguaje.

4. Ensamblaje

El **ensamblador** convierte el código en lenguaje ensamblador a **código máquina** puro (ceros y unos). Genera un archivo binario u objeto (con extensión **.o** o **.obj**) que el sistema operativo ya puede empezar a reconocer.

5. Proceso de enlace (Linking)

El **enlazador** (*linker*) une tu archivo objeto con los códigos de las bibliotecas externas que el programa necesite (por ejemplo, funciones para mostrar texto en pantalla). Crea un **archivo ejecutable** final listo para guardarse en el disco duro.

6. Carga y Ejecución en el Procesador



El sistema operativo **carga** el archivo ejecutable desde el disco hacia la memoria RAM. La **CPU** (*procesador*) toma el control mediante el ciclo de instrucción: busca la orden en la memoria, la **decodifica** y la **ejecuta** usando sus circuitos internos.

1.1.3 Ejemplos de conceptos

Usa ejemplos concretos para cada concepto, mencionando en qué fase interviene cada uno en el desarrollo de un programa (análisis léxico, sintáctico, semántico, etc.).

1. Análisis léxico

Lee el código fuente carácter por carácter y los agrupa en piezas básicas con significado llamadas Tokens. Elimina espacios en blanco y comentarios.

Ejemplos de tokens: `int`, `total`, `precio`, `float`, `input`... (lo que detecta como "palabras" la maquina)

2. Análisis Sintáctico (Parser)

Recibe la lista de tokens y comprueba si siguen las reglas gramaticales del lenguaje (que la "frase" esté bien estructurada). Si no hay errores sintácticos (como olvidar un `;`), construye el **Árbol de Sintaxis Abstracta (AST)**

AST: es un esquema o diagrama en forma de árbol que dibuja el compilador para entender cómo se relacionan las piezas (tokens) de tu código.

3. Análisis Semántico



Revisa la lógica del programa utilizando el **Sistema de Tipos** y la **Tabla de Símbolos**. Comprueba que las operaciones tengan sentido (que no intentes sumar un texto con un número) y que las variables están declaradas.

Sistemas de tipos: conjunto de reglas que un lenguaje de programación utiliza para clasificar los valores y expresiones en diferentes tipos de datos

Tabla de Símbolos: estructura de datos interna que utiliza el compilador o intérprete para almacenar y gestionar la información sobre los identificadores que aparecen en el código fuente

4. Generación y Optimización de Código Intermedio

El ordenador ya ha comprobado que el código no tiene errores y tiene el árbol (AST) montado. Ahora empieza a traducir. Pero en vez de traducirlo directamente al idioma final de tu procesador (Intel, AMD, etc.), lo traduce a un idioma neutro e intermedio, como si fuera un borrador universal.

Ej: Si escribo en Java

```
int total = precio + 10;
```

El ordenador crea unas instrucciones sencillas pasito a pasito:

1. Coge el valor de **precio**.
2. Súmale **10**.
3. Guárdalo en **total**



5. Generación de Código Máquina

Coge el "borrador" de la Fase 4 y lo traduce al idioma que entiende la CPU: ceros y unos (lenguaje máquina)

1.2 Actv 2.

Investiga y clasifica los lenguajes de programación en función de: Nivel de abstracción: alto, medio y bajo. Paradigma de programación: imperativo y declarativo. Da al menos dos ejemplos de lenguajes para cada categoría y explica brevemente por qué pertenecen a esa clasificación.

El **nivel de abstracción** indica **cuán lejos está el lenguaje del hardware del ordenador.**

- **Poca abstracción:** Estás pegado a los transistores, la memoria RAM y los 0s y 1s.
- **Mucha abstracción:** Estás muy lejos del hardware, escribiendo código con palabras en inglés que parecen frases humanas.

[CPU / Memoria RAM] — (Bajo) — (Medio) — (Alto) —► [Mente Humana]

1.2.1 Los lenguajes de Bajo Nivel hablan directamente el idioma del procesador o están a un solo paso de él.

1.No tienen funciones cómodas ni gestión de memoria automática. Tienes que decirle a la CPU qué mover exactamente a cada sitio.

- La ventaja es que van a la velocidad máxima posible y aprovechan el 100% del hardware.



- La desventaja es que son extremadamente difíciles de escribir, requieren miles de líneas para cosas simples y el código que haces para un procesador Intel no sirve para un chip de móvil (ARM).

Tipos de Bajo Nivel:

1. **Lenguaje Máquina:** Son directamente **ceros y unos (0 y 1)**. Es el único idioma que la CPU entiende de forma real.
2. **Lenguaje Ensamblador (Assembly):** Sustituye las cadenas de ceros y unos por palabras cortas llamadas *mnemónicos* (**MOV, ADD, PUSH**) para que un ser humano pueda leerlo.

Los ejemplos: Ensamblador y lenguaje máquina.

1.2.2. Lenguajes de Nivel Medio

Tienen la estructura limpia de un lenguaje moderno (bucles **for**, **if**, funciones), pero a la vez te dejan "meter la mano" en la memoria RAM directamente usando **punteros**

- La ventaja es que son muy rápidos y potentes y se usan para crear **Sistemas Operativos** (como Windows o Linux) o **motores de videojuegos**.
- La desventaja es que si cometes un error gestionando la memoria, puedes colgar el ordenador o provocar fallos graves de seguridad (*Blue Screen*, fugas de memoria).

Los ejemplos: C y C++.

1.2.3. Lenguajes de Nivel Alto

Son lenguajes de programación más cercanos al lenguaje humano y más fáciles de aprender y utilizar que los lenguajes de bajo nivel. Permiten escribir instrucciones de forma sencilla sin tener que conocer directamente cómo funciona el procesador.

Ejemplos: Python, Java, C++, JavaScript.



2. Parte II:

2.1. Actv.

A continuación, se te ofrecen descripciones de fragmentos de código. Tu tarea es identificar si cada uno corresponde a un paradigma imperativo o declarativo y justificar tu respuesta.

Son formas de indicar cómo debe funcionar un programa.

- **Paradigma imperativo:** le dices al ordenador **cómo hacer algo**, paso por paso.
Ejemplos: C, Java, Python. (**CÓMO** hacerlo).
- **Paradigma declarativo:** le dices al ordenador **qué quieres conseguir**, sin indicar exactamente todos los pasos (**QUÉ** quiero obtener).
Ejemplos: SQL, HTML, Prolog

2.1.1. Fragmento 1

Un programa recorre una lista de números sumándolos uno por uno hasta obtener el total.

Pista: Se describe cómo se realiza la suma paso a paso.

Solución: Es un paradigma imperativo dado que el programa describe cómo se hace paso a paso.



2.1.2. Fragmento 2

Una consulta a una base de datos busca empleados mayores de 30 años y devuelve solo sus nombres.

Pista: Se especifica qué resultado se quiere obtener sin detallar cómo se procesa internamente.

Solución: Paradigma declarativo puesto que se dice al programa lo que se busca.

2. 2.1.3. Fragmento 3

Un programa que calcula el factorial de un número n definiendo que el factorial de 0 es 1 y, para números mayores, multiplicando el número por el factorial del número anterior.

Pista: La lógica se define recursivamente sin especificar los pasos detallados.

Solución: Es declarativo porque explica el resultado, sin especificar los pasos que debe seguir el programa

3. 2.1.4. Fragmento 4

Un programa filtra productos con precios superiores a 10 dólares recorriendo una lista y comprobando cada producto uno por uno.

Pista: Se describen detalladamente los pasos del proceso.

Solución: Paradigma declarativo ya que el programa explica cada paso ("uno por uno") de forma detallada.



2.2. Actv 2.

En equipos de 2-3 personas, seleccionen una actividad cotidiana (preparar una receta, organizar un evento, etc.) y describan la tarea de dos maneras:

Imperativa: Indicando cada uno de los pasos detallados.

Declarativa: Describiendo únicamente el resultado final que quieren lograr.

Paradigma imperativo:

Preparar una receta diciendo los pasos uno a uno, como batir los huevos, amasar la masa, cortar piezas de fruta/verdur, etc, de forma exacta y detallada para evitar cualquier error o saltarse un paso.

Paradigma declarativo:

Decir cómo quiero una receta sin indicar cada paso, por ejemplo decir que quiero una tortilla de patata pero, sin decir cómo se hace el plato, y especificando solo dos pasos; que esté muy hecha, y que sea con o sin cebolla.

2.2.1 Comparación:

Comparen las dos descripciones y discutan las ventajas y desventajas de cada enfoque. Incluyan esta comparación en el informe.

Ambas describen algo, o parte de algo pero, el paradigma imperativo es “mejor” porque explica las cosas muy detalladamente, mientras que el paradigma declarativo sólo una parte, un detalle que hay que recalcar, siendo el resto de pasos modificables.



WEBGRAFÍA:

Codigo Fuente: [Link](#)

Apoyo informativo: [Link de las definiciones](#)

Codigo objeto: [Link](#)

Fases de un programa: [Link](#)

Paradigmas de un programa: [Link](#)

Pictografía:

Portada: [Link](#)